

VISVESVARAYA NATIONAL INSTITUTE OF TECHNOLOGY

Fake Product Identification System Using Blockchain Technology

SHELL Club — VNIT Nagpur | Summer Project 2025

Duration: 16 Weeks | Stack: Solidity · Truffle · Ganache · Web3.js · React

PHASE 1: FOUNDATIONS — BLOCKCHAIN & WEB3 BASICS (Weeks 1–4)

Goal: Build a solid foundation in blockchain theory, Ethereum architecture, Solidity programming, and the Web3 development toolchain. By the end of Phase 1, you will have deployed your first smart contract on a local Ganache blockchain and built a minimal frontend that talks to it.

Week 1: Blockchain Theory & Ethereum Architecture

- What is a blockchain? Distributed ledger, immutability, consensus mechanisms (PoW vs PoS)
- Ethereum architecture: EVM, gas, accounts (EOA vs contract), transactions, blocks
- Cryptographic primitives: keccak256 hashing, public-key cryptography, digital signatures
- Why blockchain for product authentication? The oracle problem and first-mile trust
- Survey of existing solutions: QR codes, holograms, RFID — and why they fail against cloning

Mini-Project: Write a one-page comparison: what a centralized database can do vs what a blockchain adds for supply-chain provenance. Focus on immutability, auditability, and write-authentication.

Resources:

- Ethereum Whitepaper — <https://ethereum.org/en/whitepaper/>
- Ethereum Documentation — <https://ethereum.org/en/developers/docs/>
- 3Blue1Brown — But how does bitcoin actually work? (YouTube) — <https://www.youtube.com/watch?v=bBC-nXj3Ng4>
- IBM Blockchain Explained — <https://www.ibm.com/topics/blockchain>
- Paper: Blockchain for Supply Chain — Survey — <https://arxiv.org/abs/2108.00732>

Week 2: Solidity Programming — Core Concepts

- Solidity syntax: data types, mappings, structs, arrays, enums
- Functions: visibility (public, external, internal, private), view, pure, payable
- State variables, storage vs memory vs calldata
- Events: emitting and indexing; why events are the blockchain's message bus
- Modifiers: writing reusable access-control guards (onlyOwner, whenNotPaused)
- Error handling: require(), revert(), custom errors in Solidity 0.8+

Mini-Project: Write a basic ProductRegistry.sol contract: register a product (serial number, name, price) stored in a mapping, emit a ProductRegistered event, and use a modifier to restrict registration to the contract owner only.

Resources:

- CryptoZombies — Learn Solidity interactively — <https://cryptozombies.io/>
- Solidity Official Docs — <https://docs.soliditylang.org/>
- Solidity by Example — <https://solidity-by-example.org/>
- OpenZeppelin Contracts Docs — <https://docs.openzeppelin.com/contracts/>
- Patrick Collins — Full Blockchain Course (YouTube) — <https://www.youtube.com/watch?v=gyMwXuJrbJQ>

Week 3: Truffle, Ganache, and the Development Toolchain

- Truffle Suite: project structure, truffle-config.js, compile, migrate, test commands
- Ganache: local blockchain simulator, deterministic accounts, mnemonic, gas settings
- Migration scripts: 1_initial_migration.js pattern, deployer.deploy()
- Truffle console: interactive contract interaction using await syntax
- MetaMask: importing Ganache accounts, switching networks, transaction signing
- ABI export: how web3.js discovers your contract's functions from the ABI JSON

Mini-Project: Set up Truffle + Ganache. Deploy your ProductRegistry.sol from Week 2. Interact with it via truffle console — register a product, read it back, verify the event was emitted. Import a Ganache account into MetaMask.

Resources:

- Truffle Documentation — <https://trufflesuite.com/docs/truffle/>
- Ganache Documentation — <https://trufflesuite.com/docs/ganache/>
- MetaMask Getting Started — <https://metamask.io/download/>
- Truffle Quickstart Guide — <https://trufflesuite.com/docs/truffle/quickstart/>

Week 4: Web3.js & Building a Minimal dApp Frontend

- Web3.js library: providers, web3.eth.getAccounts(), contract instantiation
- Connecting MetaMask via window.ethereum: eth_requestAccounts, eth_chainId
- Calling contract functions: .call() for reads (free), .send() for writes (gas required)
- Parsing transaction receipts and events from tx logs
- HTML + vanilla JavaScript dApp: input fields, button handlers, status messages
- http-server for local frontend serving; CORS configuration

Mini-Project: Build a minimal HTML + Web3.js frontend for your ProductRegistry contract. Show a registration form (serial, name, price) and a read form (enter serial, display product details). Show wallet address in the navbar. Deploy on Ganache and test all paths.

Resources:

- Web3.js Documentation — <https://web3js.readthedocs.io/>
- Ethereum dApp Tutorial — Buildspace — <https://buildspace.so/>
- Patrick Collins — Ethers.js / Web3.js Crash Course (YouTube) — <https://www.youtube.com/watch?v=yk-j3uOqsq0>
- Paper: Blockchain Anti-Counterfeiting Survey — IEEE — <https://ieeexplore.ieee.org/document/9788449>

PHASE 2: CORE SYSTEM DESIGN — SMART CONTRACTS (Weeks 5–8)

Goal: Design and implement the four core smart contracts of BlockVerify: RoleManager (5-role RBAC), ProductRegistry (registration, QR-salt verification, ownership lifecycle), NFTProduct (ERC-721 per unit), and RecallManager (batch

recall broadcast). By the end of Phase 2 all contracts are deployed, tested with 7+ Truffle tests, and talking to each other.

Week 5: Role-Based Access Control — RoleManager.sol

- Why RBAC matters: who should be able to register, transfer, recall, and audit?
- 5 roles: MANUFACTURER, DISTRIBUTOR, RETAILER, CONSUMER, ADMIN
- Using keccak256('ROLE_NAME') as role identifiers — cheaper than strings, safer than enums
- Mappings: address => bytes32 role; address => bool isRegistered
- assignRole(), revokeRole(), getRole() functions with onlyAdmin modifier
- Events: RoleAssigned(address, bytes32, address), RoleRevoked(address, bytes32)

Mini-Project: Implement RoleManager.sol. Write a Truffle test: assign MANUFACTURER to accounts[1], verify isManufacturer(accounts[1]) returns true, try assigning with a non-admin account and confirm it reverts.

Resources:

- OpenZeppelin AccessControl — <https://docs.openzeppelin.com/contracts/4.x/access-control>
- Solidity Mappings and Structs — <https://solidity-by-example.org/mapping/>
- Paper: RBAC in Blockchain Supply Chains — IEEE 1039222 — <https://ieeexplore.ieee.org/>

Week 6: Product Registration & Rotating QR Salt — ProductRegistry.sol (Part 1)

- ProductStatus enum: ACTIVE, RECALLED, EXPIRED, COUNTERFEIT, SOLD
- Product struct: id, serialNumber, name, batchId, productPrice, ipfsHash, manufacturer, currentOwner, registeredAt, expiresAt, status, qrSalt, pendingCommit
- registerProduct(): onlyManufacturer, checks for duplicate serial, generates initial salt using keccak256(id, block.timestamp, msg.sender, blockhash(block.number-1))
- The rotating QR salt concept: why static QR codes can be photographed and cloned
- verifyProduct(id, scannedSalt): compare scannedSalt == p.qrSalt; on mismatch → COUNTERFEIT
- Auto-expiry on read: if block.timestamp >= expiresAt → set status EXPIRED inside verifyProduct()
- Scan counter: ScanWindow struct with rolling 10-minute window; emit SuspiciousActivity if >20 scans

Mini-Project: Implement registerProduct() and verifyProduct(). Write tests: (a) register and get GENUINE with correct salt, (b) wrong salt returns COUNTERFEIT and flags permanently, (c) EVM fast-forward past expiry returns EXPIRED.

Resources:

- OpenZeppelin Pausable — <https://docs.openzeppelin.com/contracts/4.x/api/security#Pausable>
- OpenZeppelin ReentrancyGuard — <https://docs.openzeppelin.com/contracts/4.x/api/security#ReentrancyGuard>
- Solidity block.timestamp — security considerations — <https://docs.soliditylang.org/en/latest/units-and-global-variables.html>
- Paper: One-Time QR Rotating Salt — IEEE 10220784 — <https://ieeexplore.ieee.org/>

Week 7: Commit-Reveal Ownership Transfer — ProductRegistry.sol (Part 2)

- The mempool front-running problem: anyone watching pending txs can see your newOwner address
- Commit-reveal pattern: Phase 1 — submit keccak256(productId, newOwner, secret); Phase 2 — reveal plaintext
- commitTransfer(id, commitHash): store hash in pendingCommit, only current owner can commit

- `revealTransfer(id, newOwner, secret, note)`: verify keccak256 match, transfer ownership, rotate salt
- Salt rotation on transfer: `p.qrSalt = keccak256(id, newOwner, block.timestamp, block.difficulty)`
- `OwnershipEvent[]` per product: records full provenance timeline (from, to, timestamp, note)
- Anti-patterns to avoid: `tx.origin` instead of `msg.sender`; single-step transfer without commit

Mini-Project: Implement `commitTransfer()` and `revealTransfer()`. Write tests: (a) correct secret completes transfer and rotates salt, (b) wrong secret reverts with commit mismatch, (c) old QR (pre-transfer salt) returns COUNTERFEIT after transfer — the anti-cloning proof.

Resources:

- Commit-Reveal Schemes in Solidity — <https://solidity-by-example.org/hacks/front-running/>
- Paper: Commit-Reveal Anti-Front-Running — IEEE 9849430 — <https://ieeexplore.ieee.org/>
- Ethereum Mempool Explained — <https://ethereum.org/en/developers/docs/transactions/>

Week 8: ERC-721 NFT & RecallManager — NFTProduct.sol + RecallManager.sol

- ERC-721 standard: non-fungible tokens, `tokenId = productId, tokenURI → IPFS metadata`
- Why ERC-721 per physical product: cryptographic ownership proof, wallet-level composability
- `NFTProduct.sol`: inherit `OpenZeppelin ERC721`, mint on register, transfer on `revealTransfer`
- IPFS: content-addressable storage, CID as the on-chain fingerprint, metadata JSON structure
- `RecallManager.sol`: `issueBatchRecall(batchId, severity, reason, productIds[])` — Class I/II/III
- `RecallManager` calling `ProductRegistry.markRecalled()` for each product ID (try/catch per ID)
- Authorizing contract-to-contract calls: `setRecallManager()` pattern vs `msg.sender` whitelist
- CRITICAL: `msg.sender` inside `markRecalled()` is the `RecallManager` contract address, not the Admin

Mini-Project: Implement `NFTProduct.sol` and `RecallManager.sol`. Wire `setRecallManager()` in the deployment migration. Test: issue a recall for BATCH-2024-01 containing product IDs [1,2] and verify `getProduct(1).isRecalled == true`.

Resources:

- OpenZeppelin ERC-721 Documentation — <https://docs.openzeppelin.com/contracts/4.x/erc721>
- IPFS Documentation — <https://docs.ipfs.tech/>
- Paper: Recall Management in Blockchain Supply Chains — IEEE 8607403 — <https://ieeexplore.ieee.org/>
- Pinata IPFS Pinning Service — <https://pinata.cloud/>

PHASE 3: FRONTEND DEVELOPMENT — 6-PAGE dAPP (Weeks 9–12)

Goal: Build all six frontend pages of the `BlockVerify` dApp: `index.html` (landing), `manufacturer.html` (registration + QR), `verify.html` (consumer verification with 4 status cards), `transfer.html` (commit-reveal UI), `inspector.html` (walletless public inspector), and `admin.html` (role management + recall + pause). Each page connects to the live Ganache contracts via `Web3.js`.

Week 9: Architecture & Core Infrastructure — `web3-init.js`

- Event-driven MVC for dApps: contracts = model, JS modules = controller, HTML = view
- `web3-init.js`: MetaMask detection, `eth_requestAccounts`, `wallet_switchEthereumChain` (auto-switch to Chain ID 1337)
- Contract instantiation: `new web3.eth.Contract(ABI, address)` for all four contracts

- Role display in navbar: call `getRole(currentAccount)` → display Manufacturer / Admin / Consumer
- Async/await pattern for blockchain calls: always await tx receipt before updating UI
- Event-sourced state: derive UI state from contract events, not from counter variables
- `export-abis.js`: write a Node.js script that reads Truffle build artifacts and exports a `CONTRACTS` object to `frontend/js/contracts-abi.js`

Mini-Project: Build `web3-init.js` fully. Create a `seed-data.js` Truffle exec script that assigns all 4 non-admin roles and registers 3 demo products (Paracetamol 500mg, Amoxicillin 250mg, Ibuprofen 400mg) automatically on every fresh deploy.

Resources:

- Web3.js Contract Instance — <https://web3js.readthedocs.io/en/v1.10.0/web3-eth-contract.html>
- MetaMask Ethereum Provider API — <https://docs.metamask.io/wallet/reference/provider-api/>
- html5-qrcode Library (camera QR scanner) — <https://github.com/mebjas/html5-qrcode>

Week 10: Manufacturer & Verify Pages

- `manufacturer.html`: 6-field registration form (serial, name, batchId, price, expiresAt, IPFS hash)
- After tx confirm: call `getQRSalt(productId)`, generate QR using `QRCode.js` with payload `'productId|salt'`
- Download QR PNG button, Copy Salt to Clipboard button
- Date input handling: `<input type='date'>` always returns YYYY-MM-DD — use `Date.UTC()` for correct Unix timestamp parsing
- `verify.html`: `html5-qrcode` camera scanner → decode `'productId|salt'` → auto-fill fields → auto-verify
- 4 status cards: GENUINE (green), COUNTERFEIT (red), RECALLED (orange), EXPIRED (grey)
- Each card shows: product name, serial, batch, price, expiry countdown, scan count, ownership journey timeline

Mini-Project: Build `manufacturer.html` and `verify.html`. Register a product with no expiry → verify GENUINE. Verify with `salt=99999` → COUNTERFEIT. Fast-forward EVM time via `evm_increaseTime` in truffle console → verify EXPIRED.

Resources:

- `QRCode.js` Library — <https://github.com/davidshimjs/qrcodejs>
- `html5-qrcode` Scanner — <https://github.com/mebjas/html5-qrcode>
- Truffle `evm_increaseTime` — <https://trufflesuite.com/docs/truffle/how-to/debug-test/use-ganache-with-truffle/>

Week 11: Transfer, Admin & Recall Banner Pages

- `transfer.html`: Step 1 — Commit (enter `productId`, `newOwner`, note → hash stored on-chain); Step 2 — Reveal (same page, NO refresh — secret lives in `sessionStorage`)
- `admin.html`: role assignment table with quickFill buttons, Emergency Pause / Unpause, Issue Batch Recall form
- Issue Recall logic in `admin.js`: loop `getTotalProducts()`, check each `getProduct(i).batchId`, collect matching IDs array, then call `issueBatchRecall(batchId, severity, reason, productIds[])`
- `recall-banner.js`: polls `RecallManager.getPastEvents('RecallIssued')` every 8 seconds; shows sticky banner (Class I = red, Class II = orange, Class III = blue); dismiss persists in `localStorage`
- Pause test: while paused, manufacturer registration fails with `Pausable: paused`; reads still work

Mini-Project: Build `transfer.html` and `admin.html`. Run Phase 8 of the testing guide: register a product, commit transfer to Distributor, reveal transfer, verify with new salt (GENUINE), verify with old salt (COUNTERFEIT). Issue a recall and watch the orange banner appear on `verify.html` within 8 seconds.

Resources:

- OpenZeppelin Pausable — <https://docs.openzeppelin.com/contracts/4.x/api/security#Pausable>
- localStorage API — <https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage>
- Paper: Instant Recall Propagation in Blockchain — IEEE 8607403 — <https://ieeexplore.ieee.org/>

Week 12: Walletless Inspector & Integration Testing

- inspector.html: uses new Web3(new Web3.providers.HttpProvider('http://127.0.0.1:8545')) — zero MetaMask, zero gas, zero browser extensions
- Displays: product ID, serial, batch, price, manufacturer, current owner, registration date, expiry, total scans, IPFS hash
- Full ownership history timeline: OwnershipEvent[] from getHistory(id) — each entry shows from, to, timestamp, note
- Open in incognito window (Ctrl+Shift+N) to demonstrate truly walletless experience
- Integration testing: verify all 4 status cards, recall banner 8-second live test, commit-reveal anti-cloning demo, pause/unpause, scan count increment
- MetaMask troubleshooting: always reset account activity (Settings → Advanced → Clear activity and nonce data) after truffle migrate --reset

Mini-Project: Complete all 14 phases of the testing guide end-to-end. Document every test with a screenshot. Prepare the 5-minute evaluator demo script.

Resources:

- Web3.js HttpProvider (walletless) — <https://web3js.readthedocs.io/en/v1.10.0/web3.html#providers>
- Incognito Mode Testing — <https://developer.chrome.com/docs/devtools/>
- Paper: Walletless Public Inspector for Blockchain Provenance — <https://ieeexplore.ieee.org/>

PHASE 4: ADVANCED FEATURES & RESEARCH (Weeks 13–16)

Goal: Extend BlockVerify beyond the core MVP with advanced features: SuspiciousActivity event integration, AI-assisted anomaly detection on scan streams, production-grade hardening (multi-sig admin, proxy upgrade pattern), and writing a research paper targeting an IEEE conference. This phase is where your project transforms from a working demo into a publishable research contribution.

Week 13: Advanced Security — Multi-sig & Upgrade Patterns

- Named production gap: single admin key — if compromised, any batch can be recalled or fake manufacturer assigned
- Gnosis Safe multi-sig: using a 2-of-3 multisig wallet as the admin address (no single point of failure)
- OpenZeppelin TimelockController: 24-hour delay on admin actions (recall issuance, pause)
- Proxy upgrade pattern: TransparentUpgradeableProxy — deploy v2 without losing existing product records
- OpenZeppelin Upgradeable Contracts: initialize() instead of constructor(), storage layout rules
- Security audit checklist: reentrancy, integer overflow (Solidity 0.8+ handles this), access control bypass, front-running

Mini-Project: Replace the single admin address with a Gnosis Safe 2-of-3 multisig on Ganache. Test that issuing a recall from a single key fails and requires 2 confirmations. Optionally: implement a TimelockController with a 24-hour delay on issueBatchRecall().

Resources:

- Gnosis Safe Documentation — <https://docs.safe.global/>
- OpenZeppelin Upgradeable Contracts — <https://docs.openzeppelin.com/upgrades-plugins/>
- OpenZeppelin TimelockController — <https://docs.openzeppelin.com/contracts/4.x/governance>
- Smart Contract Security Best Practices — <https://consensys.github.io/smart-contract-best-practices/>
- Paper: Blockchain Security Vulnerabilities Survey — <https://arxiv.org/abs/2105.02975>

Week 14: AI Integration — Anomaly Detection on Scan Streams

- SuspiciousActivity event already emitted by ProductRegistry when >20 scans in 10 minutes
- Off-chain anomaly listener: subscribe to SuspiciousActivity events via WebSocket (Infura/Alchemy)
- Isolation Forest model: unsupervised anomaly detection on scan rate time-series data
- Feature engineering from on-chain events: scans-per-hour per productId, geographic spread (from IP logs), time-delta between consecutive scans
- Model inference: flag products where scan pattern deviates $>2\sigma$ from baseline as potential cloning attack
- Oracle pattern: anomaly score published back on-chain via an admin multisig call or Chainlink oracle
- Why on-chain ML is infeasible: gas costs for matrix operations; the correct architecture is off-chain compute + on-chain result

Mini-Project: Build a Python script that subscribes to SuspiciousActivity events from Ganache and runs an Isolation Forest on the event stream. Generate 25 rapid scans on one product and show the model flags it. Visualize the scan rate time-series with matplotlib.

Resources:

- Scikit-learn Isolation Forest — <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html>
- Web3.py — Python Ethereum library — <https://web3py.readthedocs.io/>
- Chainlink Oracles Documentation — <https://docs.chain.link/>
- Paper: AI-Powered Anti-Counterfeiting with Blockchain — <https://arxiv.org/abs/2301.09876>
- Paper: Scan Anomaly Detection in Supply Chain Blockchain — IEEE 10922890 — <https://ieeexplore.ieee.org/>

Week 15: Production Hardening & Mainnet Preparation

- Replace Ganache with a public testnet: Sepolia (Ethereum) or Mumbai (Polygon) via Infura/Alchemy
- Environment variables: store private keys and API keys in .env (never commit to Git)
- Real IPFS pinning: Pinata backend-signed upload (never expose Pinata API key in frontend)
- WebSocket recall events: replace 8-second polling with Alchemy/Infura WebSocket subscription to RecallIssued events — push, not poll
- Gas optimization techniques: tight variable packing in structs, using uint128 for scan counters, events instead of storage for history
- Hardhat as Truffle alternative: faster testing, better stack traces, TypeScript support
- Test coverage tool: solidity-coverage — ensure >80% line coverage before mainnet

Mini-Project: Deploy BlockVerify to Sepolia testnet. Register a real product. Verify it from a phone browser using the live URL. Set up a WebSocket event listener for recall broadcasts. Run solidity-coverage and document your coverage report.

Resources:

- Infura — Ethereum Node Provider — <https://infura.io/>

- Alchemy — Blockchain Developer Platform — <https://www.alchemy.com/>
- Hardhat Documentation — <https://hardhat.org/>
- solidity-coverage — <https://github.com/sc-forks/solidity-coverage>
- Pinata IPFS Pinning API — <https://pinata.cloud/>

Week 16: Research Paper Writing & Conference Submission

- Survey of 65 IEEE papers on blockchain anti-counterfeiting — build a feature matrix
- Novel contribution statement: BlockVerify is the first system to integrate 11 defences in a single Solidity stack — rotating QR salt, commit-reveal, auto-expiry on read, walletless inspector, Pausable + ReentrancyGuard — none present together in prior work
- Paper structure: Abstract, Introduction, Related Work, System Architecture, Smart Contract Design, Frontend Implementation, Security Analysis, Performance Evaluation (gas costs), Conclusion, References
- Figures to include: system architecture diagram, ProductStatus state machine, ownership transfer sequence diagram, gas benchmark table, 4 status card screenshots
- Target conferences: ICSP 2026 (<https://icsp.co.in/2026/>), MVAI 2026 (<https://2026.mvai.in/>), NGNDAI-MNNIT (<https://www.ngndai.mnnit.ac.in/>)
- LaTeX using IEEE template; Overleaf for collaborative editing

Mini-Project: Write the complete research paper (6–8 pages, IEEE format). Include gas benchmark table comparing registerProduct, verifyProduct, commitTransfer, revealTransfer, issueBatchRecall. Submit to at least one target conference and upload final code + paper to GitHub with a comprehensive README.

Resources:

- IEEE LaTeX Templates — <https://www.ieee.org/conferences/publishing/templates.html>
- Overleaf — Online LaTeX Editor — <https://www.overleaf.com/>
- ICSP 2026 Conference — <https://icsp.co.in/2026/>
- MVAI 2026 Conference — <https://2026.mvai.in/>
- NGNDAI-MNNIT Conference — <https://www.ngndai.mnnit.ac.in/>
- Paper: Blockchain Anti-Counterfeiting (Reference Benchmark) — <https://arxiv.org/abs/2108.00732>

SUBMISSION & DELIVERABLES

GitHub Repository: One repo with the full project. Required files:

- contracts/ — All four .sol files
- migrations/ — All migration scripts
- test/ — Truffle test suite (minimum 7 tests, all passing)
- scripts/ — seed-data.js, export-abis.js, verify-deployment.js
- frontend/ — All six HTML pages + JS modules
- README.md — Setup instructions, architecture overview, screenshots of all 4 status cards
- paper/ — Final IEEE-format research paper (PDF)

Minimum Passing Criteria:

- 7 Truffle tests passing (truffle test output screenshot)
- All 4 verification status cards demonstrable on live Ganache

- Recall banner appears within 10 seconds on verify.html
- Commit-reveal transfer working with anti-cloning proof (old QR → COUNTERFEIT)
- Walletless inspector.html working in incognito window
- Admin pause test: registration fails while paused

Bonus (for research paper track):

- Deployed to Sepolia testnet with live URL
- AI anomaly detection script with visualization
- Gnosis Safe multi-sig admin integration
- Conference paper submission acknowledgement

EXTRA FEATURES YOU CAN EXPLORE

These are optional extensions for students who finish the core deliverables early:

- **Batch CSV Registration:** Allow manufacturers to upload a CSV of 500+ products and register them all in a loop. Challenge: staying within block gas limit (6.7M per block on Ganache).
- **ZK-Proof of Price Range:** Use a zero-knowledge proof to prove a product's price falls within a range without revealing the exact price — so competitors can't read your pricing from the chain.
- **Cross-Chain Recall Propagation:** Use Chainlink CCIP to broadcast recall notices from Ethereum to Polygon, ensuring recalls reach all chains where products are tracked.
- **Shareable Verification Card:** After scanning a genuine product, generate a downloadable image card showing the GENUINE stamp, product name, and QR — for sharing via WhatsApp or email.
- **IoT Cold-Chain Integration:** Register temperature sensor readings as on-chain events during pharmaceutical transport. Flag a product RECALLED if temperature exceeded threshold during transit.
- **Hardhat + TypeScript Migration:** Migrate the entire project from Truffle + vanilla JS to Hardhat + TypeScript + ethers.js. Compare developer experience, test speed, and stack trace quality.